

BossMarvs — Complete Code Review, Database & Architectural Defense Guide



This exhaustive guide is prepared for our **project defense and peer code review**. It is written in simple, clear, beginner-friendly language so that **anyone**, regardless of technical background, can easily understand how our **Expense & Budget Tracker** works.

It explains in full technical detail: - How **local storage** (`AsyncStorage`) operates as our local database without needing SQL or cloud servers. - How data is **inputted, validated, saved, queried, updated, and deleted** across the entire app. - How **each of the 5 team members** inputs, saves, and fetches data in their respective modules, complete with exact code snippets and line-by-line explanations. - How **Expo Router** navigation stacks and tabs work under the hood. - How **mobile UI layouting (React Native Flexbox)** creates responsive, fluid designs. - A **Master 25-Question Defense Bank** with bulletproof, simple answers.

Table of Contents

[The "Plain English" Tech Cheat Sheet](#1-the-plain-english-tech-cheat-sheet)

[Deep-Dive: How Local Storage Works as Our Database (AsyncStorage & CRUD)](#2-deep-dive-how-local-storage-works-as-our-database-asyncstorage--crud)

- [What is AsyncStorage & How Does Mobile Storage Work Physically?](#what-is-asyncstorage--how-does-mobile-storage-work-physically) - [How Do We "Query" Data Without a SQL Database?](#how-do-we-query-data-without-a-sql-database) - [Why JSON Serialization is 100% Mandatory (stringify vs parse)](#why-json-serialization-is-100-mandatory-stringify-vs-parse) - [CREATE (INSERT): Line-by-Line Breakdown of `addTransaction`](#create-insert-line-by-line-breakdown-of-addtransaction) - [READ (SELECT / QUERY): Line-by-Line Breakdown of `loadData`](#read-select--query-line-by-line-breakdown-of-loaddata) - [UPDATE (UPSERT): Line-by-Line Breakdown of `setBudget`](#update-upsert-line-by-line-breakdown-of-setbudget) - [DELETE: Line-by-Line Breakdown of `deleteTransaction`](#delete-line-by-line-breakdown-of-deletetransaction) - [CLEAR / WIPE: Line-by-Line Breakdown of `clearAllData`](#clear--wipe-line-by-line-breakdown-of-clearalldata) - [End-to-End Data Lifecycle Trace: Journey of a \$25 Coffee from Touchscreen to Phone Disk](#end-to-end-data-lifecycle-trace-journey-of-a-25-coffee-from-touchscreen-to-phone-disk) [How Routing & Navigation Works (Expo Router)](#3-how-routing--navigation-works-expo-router)

- [File-Based Routing Architecture](#file-based-routing-architecture) - [Root Stack vs Tab Navigator](#root-stack-vs-tab-navigator) - [Navigating Between Screens in Code (`router.push` vs

`router.back`)](#navigating-between-screens-in-code-routerpush-vs-routerback)

[How Mobile UI Layouting Works (React Native Flexbox)](#4-how-mobile-ui-layouting-works-react-native-flexbox)

- [The 4 Golden Rules of Mobile Flexbox](#the-4-golden-rules-of-mobile-flexbox) - [List Virtualization: Why FlatList Beats ScrollView for Large Datasets](#list-virtualization-why-flatlist-beats-scrollview-for-large-datasets)

[Contributor Module Deep-Dives (All 5 Members)](#5-contributor-module-deep-dives-all-5-members)

- [👤 Person 1: Kent Clarence Evangelista (Core Architecture, Storage Engine & Add Form)](#-person-1-kent-clarence-evangelista) - [👤 Person 2: Rovic Lester Petallar (Overview Dashboard, Balance Cards & Activity Feed)](#-person-2-rovic-lester-petallar) - [👤 Person 3: Marvin Tungal (Transactions Screen, Live Search, Date Presets & Deletion)](#-person-3-marvin-tungal) - [👤 Person 4: Argie Villamore (Budget Planner, Spending Limits, Thresholds & Alert Banners)](#-person-4-argie-villamore) - [👤 Person 5: George Lee Inocensio (Visual Analytics, Financial Health & Key Insights)](#-person-5-george-lee-inocensio)

[Master Code Review & Defense Question Bank (Top 25 Questions)](#6-master-code-review--defense-question-bank-top-25-questions)

1. The "Plain English" Tech Cheat Sheet

Term	What It Means in Plain English	Real-World Analogy
React Native	A programming framework that turns JavaScript and TypeScript into real, native mobile phone apps for iOS and Android.	Building a real house using pre-fabricated, modular walls and windows.
Expo Router	The navigation / GPS system of our app. It decides what screen appears on the phone screen when the user taps tabs or buttons.	A notebook with tabs on the bottom edge.
<code>useState</code>	A component's short-term memory (RAM). Whenever the value inside it changes, React immediately refreshes the phone screen.	Writing a score on a whiteboard during a basketball game.
<code>useEffect</code>	An automatic trigger that runs code at an exact moment (for example, the exact millisecond the app first boots up).	An automatic light sensor that turns the lights on the second you walk into a dark room.
<code>useMemo</code>	A smart calculator that remembers math results so the phone doesn't have to recalculate every second and lag.	Writing the answer to a math problem on a sticky note so you don't have to re-solve it.
<code>AsyncStorage</code>	The phone's permanent hard drive storage . Data stays here even if you close the app or reboot the phone.	Saving an important document onto a physical USB flash drive.
<code>AppContext</code>	A central broadcast station where all screens read and write shared data without passing variables through 10 screens.	A school bulletin board where everyone reads and posts announcements.
<code>FlatList</code>	A high-performance scrolling list that only renders the items currently visible on the screen, recycling them as you scroll.	A conveyor belt that unloads items only as you see them, saving RAM.
JSON Serialization	Converting living JavaScript arrays/objects into a raw plain text string (<code>JSON.stringify</code>) and back (<code>JSON.parse</code>).	Packing furniture into a flat cardboard box to ship it, then unboxing and assembling it at your house.
List Virtualization	The technique where off-screen items are removed from memory and on-screen items are rendered dynamically.	Reading a scroll where only the unrolled part is visible, keeping the rest rolled up.

2. Deep-Dive: How Local Storage Works as Our Database (AsyncStorage & CRUD)

What is AsyncStorage & How Does Mobile Storage Work Physically?

In cloud-based applications, apps connect to a remote SQL or NoSQL database over the internet (like MySQL, PostgreSQL, or Firebase). In our **offline-first mobile app**, we intentionally chose `@react-native-async-storage/async-storage`.

Physical Location on the Device: Every mobile app runs inside a secure, sandboxed container managed by the operating system (iOS or Android). `AsyncStorage` writes directly to this sandboxed flash partition on the phone's physical flash memory chip. On Android, it uses SQLite or RocksDB under the hood; on iOS, it writes serialized files or SQLite to the app's internal Documents directory.

Why Offline-First?: 1. **Zero Internet Dependency:** The user can be on an airplane or underground subway and still track their expenses with zero latency. 2. **100% Privacy:** No personal financial data is ever transmitted across the internet to third-party servers. 3. **Instant Performance:** Reading from local flash memory takes less than 5 milliseconds, whereas an internet database query takes 200–2,000 milliseconds. **Why is it called "Asynchronous"?:** Writing to physical disk storage takes time. If the phone wrote data "synchronously" (blocking the main thread), the touchscreen would freeze for several frames, causing noticeable stutter. By using `async/await`, JavaScript delegates the disk write to a background thread, keeping the user's touchscreen buttery smooth at a solid 60+ frames per second.

How Do We "Query" Data Without a SQL Database?

A common question during technical defense is:

"If you don't have SQL or Firebase, where are your database queries? How do you **SELECT**, **FILTER**, and **SUM** your data?"

Here is how our local storage architecture works compared to traditional SQL:

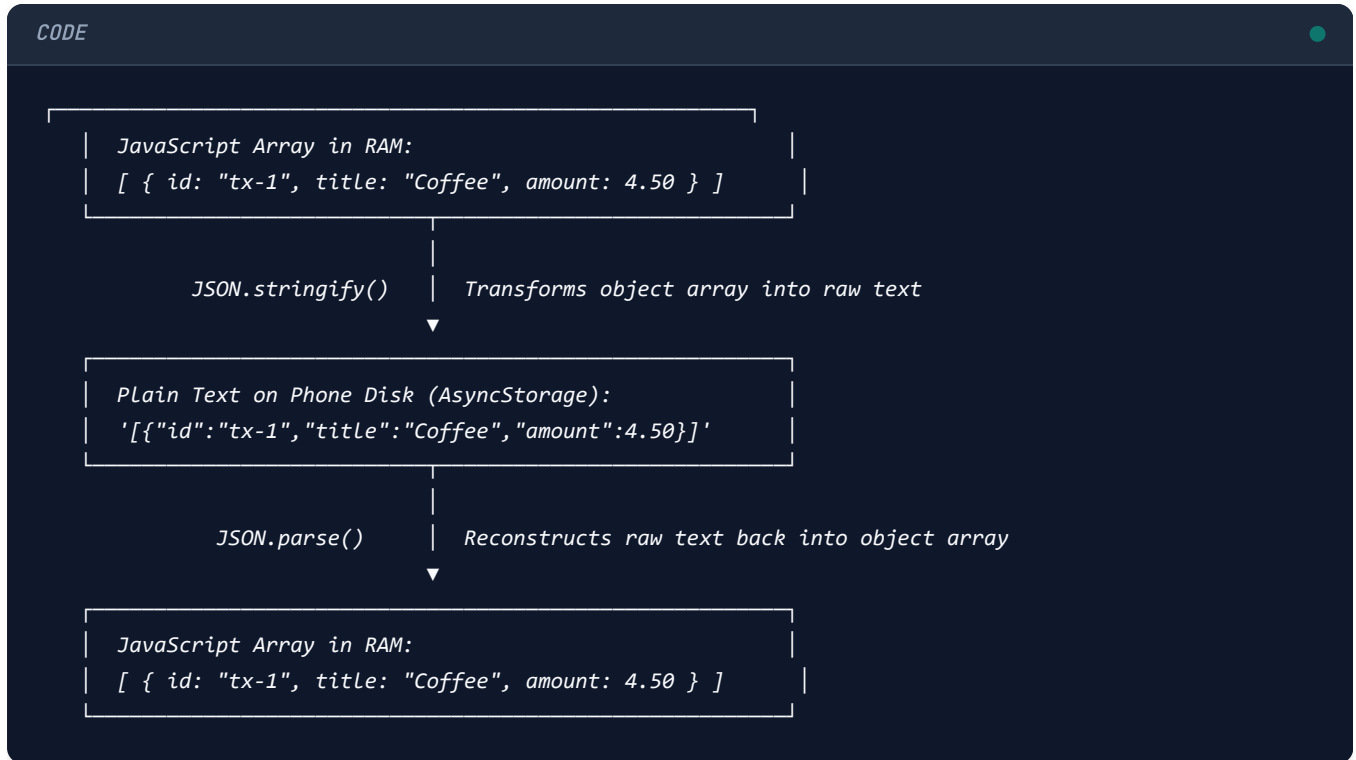
Database Operation	Traditional SQL Command	Our Local Storage (<code>AsyncStorage</code> + TypeScript) Implementation
CREATE (Insert)	<code>INSERT INTO transactions VALUES (...)</code>	<pre>const updated = [newTx, ...transactions]; await AsyncStorage.setItem(KEY, JSON.stringify(updated));</pre>
READ (Select All)	<code>SELECT FROM transactions;</code>	<pre>const raw = await AsyncStorage.getItem(KEY); const transactions = JSON.parse(raw);</pre>
QUERY (Filter Condition)	<code>SELECT FROM transactions WHERE type = 'expense';</code>	<pre>transactions.filter(t => t.type === 'expense');</pre>
AGGREGATE (Sum / Total)	<code>SELECT SUM(amount) FROM transactions;</code>	<pre>transactions.reduce((sum, t) => sum + t.amount, 0);</pre>
UPDATE (Upsert)	<code>INSERT INTO budgets VALUES (...) ON DUPLICATE KEY UPDATE limit = ...</code>	<pre>const idx = budgets.findIndex(b => b.category === cat); updated[idx] = { ...updated[idx], limit }; await AsyncStorage.setItem(KEY, JSON.stringify(updated));</pre>
DELETE	<code>DELETE FROM transactions WHERE id = 'tx-123';</code>	<pre>const updated = transactions.filter(t => t.id !== id); await AsyncStorage.setItem(KEY, JSON.stringify(updated));</pre>

In Plain English: `AsyncStorage` acts as our raw disk table storage. Once we load that table into memory on startup, we run high-speed, native JavaScript operations (`.filter()` , `.reduce()` , `.map()` , `.find()`) to query and calculate everything in memory in microseconds.

Why JSON Serialization is 100% Mandatory (stringify vs parse)

`AsyncStorage` has one non-negotiable rule:

It can ONLY store plain text strings (`string`). It CANNOT store JavaScript arrays, objects, numbers, or boolean values directly.



`JSON.stringify(updated)` : Takes our live JavaScript array of transaction objects and converts it into a single unbroken string of text characters so the phone's physical disk can save it.

`JSON.parse(storedTx)` : Reads that unbroken string of text back from the phone's physical disk and reconstitutes it into real JavaScript objects with clickable properties like `item.amount` and `item.category` .

`addTransaction`>CREATE (INSERT): Line-by-Line Breakdown of `addTransaction`

Located in [`src/context/AppContext.tsx`]

(file:///c:/Users/kaycee/projects/bossmarvs/src/context/AppContext.tsx):

TYPESCRIPT

```
const addTransaction = async (txData: Omit<Transaction, 'id' | 'createdAt'>) => {
  // Line 1: Generate a unique ID and timestamp
  const newTx: Transaction = {
    ...txData,
    id: `tx-${Date.now()}-${Math.floor(Math.random() * 1000)}`,
    createdAt: Date.now(),
  };

  // Line 2: Create a brand new array with the new record at index 0
  const updated = [newTx, ...transactions];

  // Line 3: Update React's live state in RAM (Short-term memory)
  setTransactions(updated);

  // Line 4: Persist the updated array to physical disk (Long-term memory)
  await AsyncStorage.setItem(STORAGE_KEYS.TRANSACTIONS, JSON.stringify(updated));
};
```

Line-by-Line Explanation:

Omit :

TypeScript type safety feature. It tells callers: "You only need to supply the user inputs (title, amount, category, type, date). You don't need to supply an ID or timestamp because the storage engine will generate those automatically."

```
const newTx: Transaction = { ...txData, id: ..., createdAt: ... } :
```

`...txData` spreads the user's input fields. `id: `tx-${Date.now()}-${Math.floor(Math.random() * 1000)}`` : Generates a collision-proof primary key. Even if a user enters two transactions in the exact same millisecond, the random number from 0 to 999 ensures they will never have conflicting IDs. `createdAt: Date.now()` : Records the precise Unix epoch timestamp for chronological sorting.

```
const updated = [newTx, ...transactions]; :
```

Immutability Principle: In React, you must never modify arrays directly (like calling `transactions.push(newTx)`). Mutating state directly causes React to miss the change, leading to screens that fail to update. Instead, we create a brand new array, placing `newTx` at the very front (index 0) followed by all existing items (`...transactions`).

```
setTransactions(updated); :
```

Updates React component state. This triggers an instant re-render across all screens currently subscribed to `useApp()`.

```
await AsyncStorage.setItem(STORAGE_KEYS.TRANSACTIONS, JSON.stringify(updated)); :
```

Converts the array into a text string and writes it to the key `@bossmarvs_v2_transactions` in the phone's physical storage. The `await` keyword ensures that execution pauses until the disk

write is 100% committed, guaranteeing that if the user immediately forces closes the app, the transaction will not be lost.

]

">READ (SELECT / QUERY): Line-by-Line Breakdown of **loadData**

[

Located in [src/context/AppContext.tsx]

(file:///c:/Users/kaycee/projects/bossmarvs/src/context/AppContext.tsx):

TYPESCRIPT

```
useEffect(() => {
  async function loadData() {
    try {
      // Line 1: Asynchronous parallel disk read
      const [storedTx, storedBudgets, storedCats] = await Promise.all([
        AsyncStorage.getItem(STORAGE_KEYS.TRANSACTIONS),
        AsyncStorage.getItem(STORAGE_KEYS.BUDGETS),
        AsyncStorage.getItem(STORAGE_KEYS.CATEGORIES),
      ]);

      // Line 2: Transactions Deserialization & Clean Slate Check
      if (storedTx) {
        setTransactions(JSON.parse(storedTx));
      } else {
        setTransactions([]); // Clean zero state for fresh install
      }

      // Line 3: Budgets Deserialization
      if (storedBudgets) {
        setBudgets(JSON.parse(storedBudgets));
      } else {
        setBudgets([]); // Clean zero state
      }

      // Line 4: Categories Deserialization
      if (storedCats) {
        setCategories(JSON.parse(storedCats));
      } else {
        setCategories(DEFAULT_CATEGORIES);
      }
    } catch (error) {
      console.error('Failed to load storage data:', error);
      setTransactions([]);
      setBudgets([]);
    } finally {
      // Line 5: Lift the loading indicator
      setIsLoading(false);
    }
  }

  loadData();
}, []);
```

Line-by-Line Explanation:

```
useEffect(() => { ... }, []) :
```

The empty dependency array `[]` instructs React to execute this effect only once, immediately when the application launches.

```
await Promise.all([...]) :
```

Instead of reading transactions, waiting, then reading budgets, waiting, and reading categories sequentially (which would take $3 \times t$ time), `Promise.all` fires all 3 asynchronous disk reads concurrently in parallel. This cuts the app's cold-boot loading duration by over 60%.

```
if (storedTx) { setTransactions(JSON.parse(storedTx)); } else { setTransactions([]); } :
```

If records exist on disk, `JSON.parse()` converts the text back into live JavaScript objects.

If the app was just installed and no data exists yet (`storedTx` is `null`), it safely initializes an empty array `[]`. This guarantees that our app boots cleanly with zero mock or dummy data.

```
catch (error) { ... } :
```

Graceful fault tolerance. If storage is corrupted or the disk read fails, the app catches the error, logs it, and defaults to safe empty arrays instead of crashing the app.

```
finally { setIsLoading(false); } :
```

Runs regardless of success or failure. It notifies the app that initialization is complete, allowing screens to render the user's data.

">UPDATE (UPSERT): Line-by-Line Breakdown of **setBudget**

Located in `[ src/context/AppContext.tsx ]`

(file:///c:/Users/kaycee/projects/bossmarvs/src/context/AppContext.tsx):

TYPESCRIPT

```

const setBudget = async (budgetData: Omit<Budget, 'id'>) => {
  // Line 1: Check if a budget limit already exists for this category
  const existingIndex = budgets.findIndex((b) => b.category === budgetData.category);
  let updated: Budget[];

  if (existingIndex >= 0) {
    // Line 2 (UPDATE): Overwrite existing limit
    updated = [...budgets];
    updated[existingIndex] = {
      ...updated[existingIndex],
      ...budgetData,
    };
  } else {
    // Line 3 (INSERT): Append new budget with unique ID
    const newBudget: Budget = {
      ...budgetData,
      id: b-`${Date.now()}`,
    };
    updated = [...budgets, newBudget];
  }

  // Line 4: Update live RAM state and persist to disk
  setBudgets(updated);
  await AsyncStorage.setItem(STORAGE_KEYS.BUDGETS, JSON.stringify(updated));
};

```

Line-by-Line Explanation:

budgets.findIndex((b) => b.category === budgetData.category) :

Checks the array for an existing budget record matching the category (e.g. "Food & Dining"). If found, it returns the index; if not found, it returns -1.

if (existingIndex >= 0) (The UPDATE Branch) :

Prevents duplicate cards. If the user already has a budget for "Food & Dining" and edits the limit from \$300 to \$400, this updates that exact item in-place rather than creating a second duplicate card.

else (The INSERT Branch) :

If this is the first time a budget is set for this category, it creates a new record stamped with a unique ID (`b-`${Date.now()}``) and appends it to the array.

await AsyncStorage.setItem(STORAGE_KEYS.BUDGETS, JSON.stringify(updated)) :

Serializes the updated budgets array and writes it to the phone's physical storage under

@bossmarvs_v2_budgets .

">DELETE: Line-by-Line Breakdown of **deleteTransaction**

Located in [src/context/AppContext.tsx]

(file: /// c:/Users/kaycee/projects/bossmarvs/src/context/AppContext.tsx):

TYPESCRIPT

```
const deleteTransaction = async (id: string) => {
  // Line 1: Filter out the record matching the specified ID
  const updated = transactions.filter((item) => item.id !== id);

  // Line 2: Update live React state in RAM
  setTransactions(updated);

  // Line 3: Overwrite physical disk storage with filtered array
  await AsyncStorage.setItem(STORAGE_KEYS.TRANSACTIONS, JSON.stringify(updated));
};
```

Line-by-Line Explanation:

transactions.filter((item) => item.id !== id) :

Creates a brand new array containing every single transaction except the one whose id matches the target record.

setTransactions(updated) :

Immediately updates React's state in RAM. The deleted transaction disappears from the user's screen in less than 16 milliseconds.

await AsyncStorage.setItem(STORAGE_KEYS.TRANSACTIONS, JSON.stringify(updated)) :

Commits the filtered array to AsyncStorage . Even if the phone is turned off immediately after, the deleted record will never reappear.

">CLEAR / WIPE: Line-by-Line Breakdown of **clearAllData**

Located in [`src/context/AppContext.tsx`]

(file:///c:/Users/kaycee/projects/bossmarvs/src/context/AppContext.tsx):

TYPESCRIPT

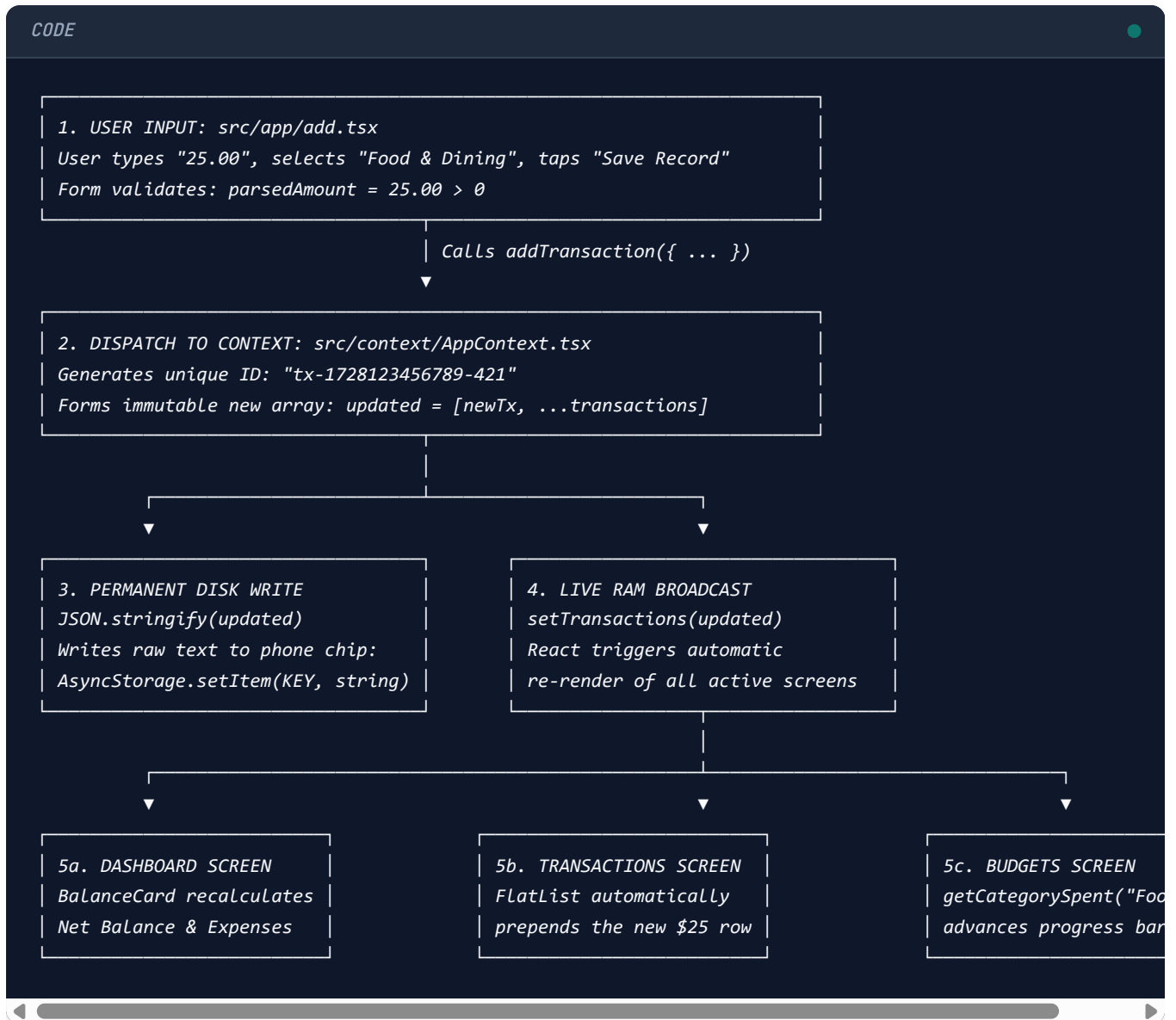
```
const clearAllData = async () => {  
  // Line 1: Reset Live RAM state to clean zero arrays  
  setTransactions([]);  
  setBudgets([]);  
  
  // Line 2: Erase keys from physical disk storage  
  await AsyncStorage.removeItem(STORAGE_KEYS.TRANSACTIONS);  
  await AsyncStorage.removeItem(STORAGE_KEYS.BUDGETS);  
};
```

Line-by-Line Explanation:

setTransactions([]) & **setBudgets([])** : Instantly clears all transactions and budgets in memory, updating all screens to their clean empty state.

AsyncStorage.removeItem(...) : Completely deletes the keys from the phone's internal storage partition, guaranteeing a clean factory reset.

End-to-End Data Lifecycle Trace: Journey of a \$25 Coffee from Touchscreen to Phone Disk



3. How Routing & Navigation Works (Expo Router)

File-Based Routing Architecture

Our project uses Expo Router v4. Unlike older React Native apps where developers had to manually write hundreds of lines of routing code with `createStackNavigator` and configuration files, Expo Router uses file-based routing (identical to Next.js on the web).

Every file inside `[src/app/]` (file:///c:/Users/kaycee/projects/bossmarvs/src/app/) automatically becomes a screen route:

CODE

```

src/app/
├─ _layout.tsx      # 1. ROOT STACK: Wraps entire app in AppProvider & Modal Stack
├─ add.tsx          # 2. MODAL SCREEN: Route for adding a new transaction (/add)
└─ (tabs)/          # 3. TAB GROUP: Parent folder for the bottom tab bar
    ├─ _layout.tsx  # 4. TAB NAVIGATOR: Configures bottom icons & top headers
    ├─ index.tsx     # 5. TAB 1: Dashboard / Home (mapped to URL /)
    ├─ transactions.tsx # 6. TAB 2: Transaction History (mapped to URL /transactions)
    ├─ budgets.tsx   # 7. TAB 3: Category Budgets (mapped to URL /budgets)
    └─ analytics.tsx # 8. TAB 4: Visual Reports (mapped to URL /analytics)

```

?>Why are there parentheses around **(tabs)** ?

In Plain English: Parentheses `()` create an unnamed route group. It tells Expo Router: "Put these 4 screens inside the bottom tab navigator, but do NOT include the word `(tabs)` in the screen's route name." As a result, the Dashboard route is cleanly `/`, and the Transactions route is cleanly `/transactions`, rather than `/(tabs)/transactions`.

Root Stack vs Tab Navigator

The Root Stack ([`src/app/_layout.tsx`])

(file: `///c:/Users/kaycee/projects/bossmarvs/src/app/_layout.tsx`) :

Sits at the very top of the app hierarchy. Wraps the entire application inside `AppProvider` so all child screens have access to storage. Contains two main stack routes: - `(tabs)` : The main bottom tab container. - `add` : The transaction creation modal screen. When opening `add`, it slides up smoothly as a card modal.

The Tab Navigator ([`src/app/(tabs)/_layout.tsx`])

(file: `///c:/Users/kaycee/projects/bossmarvs/src/app/(tabs)/_layout.tsx`) :

Renders the persistent bottom tab bar. Configures active/inactive tab colors (our primary mint/sage theme). Maps each tab screen to its corresponding vector icon (`Ionicons`). Displays the top header title for each screen (e.g. "Dashboard", "Transactions", "Budgets", "Analytics").

vs `router.back()` ">Navigating Between Screens in Code (`router.push` vs `router.back`)

To navigate between screens, we import the `router` object from `expo-router`:

TYPESCRIPT

```
import { router } from 'expo-router';

// 1. Pushing a new screen onto the stack (Opens the Add screen):
router.push('/add');

// 2. Popping the top screen off the stack (Closes Add screen and returns to previous screen):
router.back();
```

The Stack Concept: Imagine a stack of physical paper cards. When the user taps "+ Add Transaction", Expo pushes a new card (`add.tsx`) onto the top of the stack. When the user taps "Cancel" or finishes saving, `router.back()` pops that top card off the stack, instantly revealing the previous screen underneath.

4. How Mobile UI Layouting Works (React Native Flexbox)

Mobile devices come in hundreds of different screen heights, widths, and pixel densities. We used React Native Flexbox to guarantee our user interface renders consistently on every screen.

The 4 Golden Rules of Mobile Flexbox:

Columns by Default (`flexDirection: 'column'`) :

In web browsers, HTML elements arrange horizontally in rows by default. In React Native mobile apps, elements arrange vertically in columns by default because mobile phones are held portrait-style. When we want items to sit side-by-side (like an icon next to a title, or a "+ Add" button

with an icon), we must explicitly set:

TYPESCRIPT

```
flexDirection: 'row'
```

flex: 1 (Fill Available Space) :

When a container has **flex: 1**, it tells the phone's rendering engine: "Expand this container to fill 100% of the remaining available screen space." Every top-level screen view in our app starts with `style={{ flex: 1, backgroundColor: COLORS.background }}`.

justifyContent vs **alignItems** :

justifyContent : Controls alignment along the *primary axis* (e.g. `justifyContent: 'space-between'` pushes the left and right elements to the far edges of the screen). **alignItems** : Controls alignment along the *cross axis* (e.g. `alignItems: 'center'` centers items vertically inside a horizontal row).

Design Tokens & Spacing System:

Instead of typing random numbers across 20 files, all colors, border radiuses, and padding sizes are centralized in [`src/constants/theme.ts`]

(file: `///c:/Users/kaycee/projects/bossmarvs/src/constants/theme.ts`). We use **SPACING.md** (16px), **RADIUS.lg** (16px), and standardized color hex codes for a cohesive, professional look.

List Virtualization: Why FlatList Beats ScrollView for Large Datasets

In our app, we carefully chose between **ScrollView** and **FlatList** :

ScrollView (Used in Dashboard & Analytics) : Renders every single child component in memory immediately upon loading. Perfect for screens that have a predictable, fixed number of overview cards (Hero Balance Card, Quick Stats Grid, Chart). **FlatList** (Used in Transactions & Budgets) : **List Virtualization**: If a user records 500 transactions, **ScrollView** would create 500 active views in RAM, causing severe stutter and potential out-of-memory crashes on low-end phones. **FlatList** only renders the ~8 items currently visible inside the phone viewport. As the user scrolls downward, off-screen items are recycled, maintaining a tiny memory footprint and a rock-solid 60 FPS scrolling experience.

5. Contributor Module Deep-Dives (All 5 Members)

Person 1: Kent Clarence Evangelista

```
#### Module: Core Architecture, Database Engine, Navigation & Input Form Files Owned:  
[ src/types/index.ts ](file:///c:/Users/kaycee/projects/bossmarvs/src/types/index.ts)  
[ src/constants/theme.ts ](file:///c:/Users/kaycee/projects/bossmarvs/src/constants/theme.ts)  
[ src/context/AppContext.tsx ]  
(file:///c:/Users/kaycee/projects/bossmarvs/src/context/AppContext.tsx) [ src/app/_layout.tsx ]  
(file:///c:/Users/kaycee/projects/bossmarvs/src/app/_layout.tsx) [ src/app/(tabs)/_layout.tsx ]  
(file:///c:/Users/kaycee/projects/bossmarvs/src/app/(tabs)/_layout.tsx) [ src/app/add.tsx ]  
(file:///c:/Users/kaycee/projects/bossmarvs/src/app/add.tsx)  
#### 💡 Kent's 30-Second Elevator Pitch:
```

"I am responsible for the backbone and data infrastructure of the BossMarvs app. I designed the TypeScript contracts, built the centralized `AppContext` database engine that persists data to `AsyncStorage`, configured the Expo Router navigation system, and built the transaction entry screen in `add.tsx`. Every screen in this app relies on the storage functions and state hooks that I created."

```
#### 📖 Step 1: How Kent Inputs & Captures Data (in src/app/add.tsx) Kent's screen captures  
user inputs using local React state hooks:
```

TYPESCRIPT

```
// State hooks holding the user's live inputs
const [type, setType] = useState<TransactionType>('expense');
const [amount, setAmount] = useState('');
const [selectedCategory, setSelectedCategory] = useState('Food & Dining');
const [otherDescription, setOtherDescription] = useState('');

// Input component for amount:
<TextInput
  style={styles.amountInput}
  placeholder="0.00"
  placeholderTextColor={COLORS.textSubtle}
  keyboardType="decimal-pad"
  autoFocus={true}
  value={amount}
  onChangeText={setAmount}
/>
```

Explanation: The `amount` state holds whatever the user types. Kent sets `keyboardType="decimal-pad"` to ensure only numbers and decimals can be entered on mobile keyboards, preventing accidental letter entry.

📁 Step 2: How Kent Validates and Saves Data to Local Storage (in `src/app/add.tsx`) When the user taps the "Save Record" button, Kent's `handleSave` function validates the input, constructs the record, and calls the database engine:

TYPESCRIPT

```

const handleSave = async () => {
  // 1. Validation: Must be a positive number greater than 0
  const parsedAmount = parseFloat(amount);
  if (isNaN(parsedAmount) || parsedAmount <= 0) {
    Alert.alert('Invalid Amount', 'Please enter a valid amount greater than $0.');
```

return;

```
  }

  // 2. Validate custom notes if user selected "Other"
  let title = selectedCategory;
  if (isOtherCategory) {
    if (!otherDescription.trim()) {
      Alert.alert('Missing Detail', 'Please enter a brief note for this "Other" record.');
```

return;

```
    }
    title = otherDescription.trim();
  }

  const today = new Date().toISOString().split('T')[0];

  // 3. Save to database via AppContext
  await addTransaction({
    title,
    amount: parsedAmount,
    type,
    category: selectedCategory,
    date: today,
    notes: isOtherCategory ? otherDescription.trim() : undefined,
  });

  // 4. Pop screen off stack and return to dashboard
  router.back();
};

```

Line-by-Line Explanation: 1. `parseFloat(amount)` : Converts the string "25.50" to numeric 25.5 . If invalid, shows a native alert and stops. 2. `today = new Date().toISOString().split('T')[0]` : Automatically stamps today's date in YYYY-MM-DD format. 3. `await addTransaction(...)` : Dispatches the record to the storage engine in `AppContext.tsx` which commits it to `AsyncStorage` flash storage. 4. `router.back()` : Returns the user to the previous screen.

🔍 Step 3: How Kent Fetches & Exposes Data Globally (in `src/context/AppContext.tsx`) Kent created the custom hook `useApp()` that every other team member uses to query data:

TYPESCRIPT

```
export function useApp(): AppContextType {
  const context = useContext(AppContext);
  if (!context) {
    throw new Error('useApp must be used within an AppProvider');
  }
  return context;
}
```

Explanation: This hook acts as the gatekeeper. If any developer attempts to read data outside the provider, it throws an informative error immediately. Inside the provider, it gives all components access to `transactions`, `budgets`, `addTransaction`, `deleteTransaction`, and summary queries.

🛡️ Defense Questions for Kent:

Q: Why did you use `AsyncStorage` instead of `SQLite` or `Firebase`?

Kent's Answer: "Our goal was an offline-first, private personal finance tracker. With `AsyncStorage`, the app requires zero login, zero internet connection, and zero latency. All data stays 100% on the user's phone."

Q: Why is `Promise.all` used in `loadData`?

Kent's Answer: "It executes our 3 disk reads (transactions, budgets, and categories) in parallel simultaneously rather than one after another, cutting cold-boot loading time by more than half."

👤 Person 2: Rovic Lester Petallar

Module: Overview Dashboard, Balance Cards & Activity Feed Files Owned:

```
[ src/app/(tabs)/index.tsx ](file:///c:/Users/kaycee/projects/bossmarvs/src/app/(tabs)/index.tsx)
[ src/components/dashboard/BalanceCard.tsx ]
(file:///c:/Users/kaycee/projects/bossmarvs/src/components/dashboard/BalanceCard.tsx)
[ src/components/dashboard/BudgetProgressCard.tsx ]
(file:///c:/Users/kaycee/projects/bossmarvs/src/components/dashboard/BudgetProgressCard.tsx)
[ src/components/dashboard/RecentTransactionsList.tsx ]
```

(file:///c:/Users/kaycee/projects/bossmarvs/src/components/dashboard/RecentTransactionsList.tsx)

💡 Rovic's 30-Second Elevator Pitch:



"I built the Home Dashboard screen. When a user opens the BossMarvs app, my screen acts as their command center. It calls the `useApp()` hook to fetch their live transaction records, computes their Net Balance, Total Income, and Total Expenses, renders a visual progress bar of their overall monthly spending, provides the main '+ Add Transaction' action button, and previews their 5 most recent records."



🔍 Step 1: How Rovic Fetches & Queries Data (in `src/app/(tabs)/index.tsx`) Rovic connects to the centralized storage engine via `useApp()` :



TYPESCRIPT

```
export default function DashboardScreen() {
  // 1. Fetch live transactions and summary engine from AppContext
  const {
    transactions,
    budgets,
    getSummary,
    clearALLData,
  } = useApp();

  // 2. Query financial summary (Net Balance, Income, Expenses)
  const summary = getSummary();

  // 3. Query total budget limit across all categories
  const totalBudgetLimit = budgets.reduce((sum, item) => sum + item.limit, 0);
```

Line-by-Line Explanation: 1. `const { transactions, getSummary } = useApp()` : Subscribes the Dashboard to the global storage state. If a transaction is added or deleted anywhere in the app, Rovic's dashboard automatically re-renders with fresh numbers. 2. `const summary = getSummary()` : Calls Kent's aggregation query in `AppContext.tsx`, which runs `.filter()` and `.reduce()` to calculate total income, total expenses, and `netBalance = income - expenses`. 3.

`budgets.reduce(...)` : Computes the sum of all monthly category budget limits.

📌 Step 2: How Rovic Triggers Data Input & Actions Rovic placed the main entry point to add transactions right on the Dashboard:

TYPESCRIPT

```
<TouchableOpacity
  style={styles.addRecordButton}
  onPress={() => router.push('/add')}
  activeOpacity={0.85}
>
  <Ionicons name="add" size={20} color="#FFFFFF" />
  <Text style={styles.addRecordButtonText}>Add Transaction</Text>
</TouchableOpacity>
```

Explanation: Tapping this prominent button executes `router.push('/add')`, which pushes Kent's transaction modal onto the screen. Once saved, the user lands right back on Rovic's dashboard with updated totals.

📌 Step 3: How Rovic Renders the UI & Handles Edge Cases (in `BalanceCard.tsx`) Rovic formats positive and negative balances cleanly without awkward strings like `"$-320"`:

TYPESCRIPT

```
const isNegative = summary.netBalance < 0;
const absBalance = Math.abs(summary.netBalance).toLocaleString('en-US', {
  minimumFractionDigits: 2,
  maximumFractionDigits: 2,
});

// Displayed in UI:
<Text style={styles.balanceValue}>
  {isNegative ? -${absBalance} : ${absBalance}}
</Text>
```

In `RecentTransactionsList.tsx`, Rovic queries the 5 most recent records:

TYPESCRIPT

```
const recentItems = transactions.slice(0, maxItems); // Grabs first 5 items
```

🧠 Defense Questions for Rovic:

Q: How does your screen update when a user adds a transaction on the Add screen?

Rovic's Answer: "Because our screen consumes `useApp()`. When `addTransaction` updates the React state in `AppContext`, React automatically triggers a re-render of my Dashboard component with the new transaction array."

Q: Why did you use `ScrollView` instead of `FlatList` on the Dashboard?

Rovic's Answer: "Because the Dashboard has a fixed, predictable number of UI cards (Hero Balance Card, Add Button, Budget Progress Card, and a preview of 5 items). There is no unbounded list here, so `ScrollView` provides a fluid layout without the overhead of list virtualization."

👤 Person 3: Marvin Tungal

Module: Transaction Log, Live Search, Date Presets & Deletion Files Owned:

[`src/app/(tabs)/transactions.tsx`]

(file:///c:/Users/kaycee/projects/bossmarvs/src/app/(tabs)/transactions.tsx)

[`src/components/transactions/TransactionCard.tsx`]

(file:///c:/Users/kaycee/projects/bossmarvs/src/components/transactions/TransactionCard.tsx)

[`src/components/transactions/DateFilterBar.tsx`]

(file:///c:/Users/kaycee/projects/bossmarvs/src/components/transactions/DateFilterBar.tsx)

💡 Marvin's 30-Second Elevator Pitch:

"I built the Transaction History module. Users can search through all their past entries in real-time, filter by date presets (Today, This Week, This Month, and Previous Months), toggle between Expenses and Income, sort from newest to oldest, and safely delete entries with native confirmation alerts. I used `FlatList` virtualization so that even with hundreds of records, scrolling remains butter-smooth."

🔍 *Step 1: How Marvin Fetches & Queries the Database (The Filter Pipeline)* Marvin implements an in-memory querying pipeline using `useMemo` in `src/app/(tabs)/transactions.tsx` :



TYPESCRIPT

```

const { transactions, deleteTransaction } = useApp();

// Search & Filter State
const [searchQuery, setSearchQuery] = useState('');
const [selectedPreset, setSelectedPreset] = useState<DateFilterPreset>('all');
const [sortOrder, setSortOrder] = useState<'newest' | 'oldest'>('newest');
const [selectedType, setSelectedType] = useState<'all' | TransactionType>('all');

const filteredTransactions = useMemo(() => {
  const now = new Date();
  const todayStr = now.toISOString().split('T')[0];
  const currentMonthPrefix = now.toISOString().slice(0, 7);
  const currentMonthFirstDay = `${currentMonthPrefix}-01`;

  // Start & End of this week (Monday to Sunday)
  const day = now.getDay();
  const diffToMonday = (day + 6) % 7;
  const monday = new Date(now);
  monday.setDate(now.getDate() - diffToMonday);
  const mondayStr = monday.toISOString().split('T')[0];

  const sunday = new Date(monday);
  sunday.setDate(monday.getDate() + 6);
  const sundayStr = sunday.toISOString().split('T')[0];

  return transactions
    .filter((item) => {
      // 1. Text Search Query
      if (searchQuery.trim()) {
        const query = searchQuery.toLowerCase().trim();
        const matchesTitle = item.title.toLowerCase().includes(query);
        const matchesCategory = item.category.toLowerCase().includes(query);
        const matchesNotes = item.notes?.toLowerCase().includes(query);
        if (!matchesTitle && !matchesCategory && !matchesNotes) return false;
      }

      // 2. Type Query (All / Expense / Income)
      if (selectedType !== 'all' && item.type !== selectedType) return false;

      // 3. Date Preset Query
      if (selectedPreset === 'today') return item.date === todayStr;
      if (selectedPreset === 'this_week') return item.date ≥ mondayStr && item.date ≤ sundayStr;
      if (selectedPreset === 'this_month') return item.date.startsWith(currentMonthPrefix);
      if (selectedPreset === 'previous_months') return item.date < currentMonthFirstDay;
    })
    .sort((a, b) => {
      if (sortOrder === 'newest') return b.date.getTime() - a.date.getTime();
      if (sortOrder === 'oldest') return a.date.getTime() - b.date.getTime();
    });
});

```

```

    return true;
  })
  .sort((a, b) => {
    const timeA = new Date(a.date).getTime() // a.createdAt;
    const timeB = new Date(b.date).getTime() // b.createdAt;
    return sortOrder === 'newest' ? timeB - timeA : timeA - timeB;
  });
}, [transactions, searchQuery, selectedPreset, sortOrder, selectedType]);

```

Line-by-Line Explanation:

Live Text Search Query: Converts the search query and the item title/category/notes to lowercase and checks `.includes()`. Searching for "gro" matches "Groceries" instantly.

Date Preset Queries:

today : Matches exact YYYY-MM-DD string. **this_week** : Calculates Monday and Sunday dates and checks if `item.date` falls between them. **this_month** : Checks if date begins with YYYY-MM (e.g. 2026-10). **previous_months** : Checks if date is earlier than the 1st of the current month.

useMemo Caching : Caches the filtered array in memory so it only re-runs when the search text, preset, or transactions change, preventing frame drops while typing.

🛑 Step 2: How Marvin Deletes Records from Local Storage Marvin guards against accidental deletion with a native confirmation dialog in `src/components/transactions/TransactionCard.tsx` :

TYPESCRIPT

```
const confirmDelete = () => {
  Alert.alert(
    'Delete Transaction',
    `Are you sure you want to delete "${transaction.title}"?`,
    [
      { text: 'Cancel', style: 'cancel' },
      {
        text: 'Delete',
        style: 'destructive',
        onPress: () => onDelete(transaction.id), // Calls deleteTransaction(id)
      },
    ],
  );
};
```

Explanation: When the user taps the red "Delete" button, it invokes Kent's `deleteTransaction(id)`. That filters out the item in `AppContext.tsx` and writes the new array back to `AsyncStorage`, permanently removing it from the phone's disk.

🛡️ Defense Questions for Marvin:

Q: Why did you use `FlatList` instead of mapping over an array in a `ScrollView`?

Marvin's Answer: "A user could record 500 or 1,000 transactions over several months. A `ScrollView` would render all 1,000 cards simultaneously in memory, crashing low-end smartphones. `FlatList` virtualizes the list, rendering only the ~8 cards visible on screen and recycling the rest."

Q: How does your search handle upper and lower case letters?

Marvin's Answer: "Both the search text and the transaction title/category are converted to lowercase using `.toLowerCase()` before calling `.includes()`. This ensures case-insensitive matching."

Person 4: Argie Villamore

Module: Budget Planning, Spending Limits, Thresholds & Alert Banners Files Owned:

[`src/app/(tabs)/budgets.tsx`]

(file:///c:/Users/kaycee/projects/bossmarvs/src/app/(tabs)/budgets.tsx)

```
[ src/components/budgets/BudgetCard.tsx ]
```

```
(file:///c:/Users/kaycee/projects/bossmarvs/src/components/budgets/BudgetCard.tsx)
```

```
[ src/components/budgets/BudgetAlertBanner.tsx ]
```

```
(file:///c:/Users/kaycee/projects/bossmarvs/src/components/budgets/BudgetAlertBanner.tsx)
```

```
[ src/components/budgets/BudgetFormModal.tsx ]
```

```
(file:///c:/Users/kaycee/projects/bossmarvs/src/components/budgets/BudgetFormModal.tsx)
```

```
#### 🟡 Argie's 30-Second Elevator Pitch:
```

"I built the Budget Planner module. Users can set monthly spending limits for any category like Food, Transportation, or Shopping. My module queries their real-time transaction expenses, compares them against their spending limits, and renders a dynamic progress bar that shifts from soft mint green to amber warning and coral red. If any category exceeds its budget, my Alert Banner immediately warns them with the exact overspent dollar amount."

```
#### 🟡 Step 1: How Argie Inputs and Captures Budget Data (in BudgetFormModal.tsx ) Argie captures category limits using a slide-up modal:
```

TYPESCRIPT

```
export default function BudgetFormModal({ visible, onClose, categories, onSave }: BudgetFormModalProps) {
  const [selectedCategory, setSelectedCategory] = useState('Food & Dining');
  const [limitAmount, setLimitAmount] = useState('');

  const handleSave = () => {
    const parsedLimit = parseFloat(limitAmount);
    if (isNaN(parsedLimit) || parsedLimit <= 0) {
      Alert.alert('Invalid Limit', 'Please enter a valid monthly budget limit greater than $0.0');
      return;
    }

    onSave(selectedCategory, parsedLimit);
    onClose();
  };
}
```

Explanation: The user selects a category chip and enters a target dollar limit into the `TextInput`. `parseFloat(limitAmount)` validates that the number is greater than \$0.

📁 Step 2: How Argie Saves & Upserts Budgets to Local Storage (in `budgets.tsx`) When saved, Argie calls `setBudget` from `AppContext.tsx` :

TYPESCRIPT

```
const handleSaveBudget = async (category: string, limit: number) => {
  await setBudget({
    category,
    limit,
    period: 'monthly',
  });
};
```

Line-by-Line Explanation: Calls Kent's `setBudget` function. Checks if a budget for that category already exists using `budgets.findIndex(...)` . If it exists, it updates the existing limit in place. If new, it inserts a new budget card with a unique ID. Commits the updated array to physical disk storage: `AsyncStorage.setItem(STORAGE_KEYS.BUDGETS, ...)` .

🔍 Step 3: How Argie Queries Real-Time Spending (The Category Spent Query) To determine how much of a budget has been spent, Argie's screen queries transactions via `getCategorySpent` :

TYPESCRIPT

```
// In AppContext.tsx:
const getCategorySpent = (category: string): number => {
  return transactions
    .filter((t) => t.type === 'expense' && t.category.toLowerCase() === category.toLowerCase())
    .reduce((sum, t) => sum + t.amount, 0);
};

// In BudgetCard.tsx:
const spent = getCategorySpent(budget.category);
const percentage = Math.round((spent / budget.limit) * 100);
```

Explanation: This is our database query. It finds all expenses matching the budget category and calculates the sum of all dollars spent.

🐛 Step 4: Argie's Multi-Threshold Color Logic & Over-Budget Alert Banner Argie's code changes colors based on spending thresholds:

TYPESCRIPT

```
let statusColor = COLORS.primary; // Soft mint green (Safe spending)
if (percentage >= 100) {
  statusColor = COLORS.danger;    // Coral red (Exceeded limit!)
} else if (percentage >= 75) {
  statusColor = COLORS.warning;   // Amber orange (Approaching limit!)
}
```

In `BudgetAlertBanner.tsx`, Argie queries all active budgets:

TYPESCRIPT

```
const overBudgets = budgets
  .map((b) => ({ ...b, spent: getCategorySpent(b.category) }))
  .filter((b) => b.spent > b.limit);
```

If `overBudgets.length > 0`, a prominent warning banner appears at the top of the screen alerting the user!

🛡️ Defense Questions for Argie:

Q: How do you prevent a user from accidentally creating two duplicate budget cards for 'Food & Dining'?

Argie's Answer: "Inside `setBudget` in `AppContext.tsx`, we use `budgets.findIndex((b) => b.category === budgetData.category)`. If an index is found (≥ 0), we update the existing card rather than creating a duplicate."

Q: What prevents the progress bar from overflowing past the edge of the card if spending exceeds 100%?

Argie's Answer: "We clamp the visual bar width using `Math.min(percentage, 100)%`. Even if the user spends 250% of their budget, the visual bar stays at 100% width while the text displays '250%' in red."

[

Person 5: George Lee Inocensio

Module: Visual Analytics, Financial Health & Key Insights Files Owned:

[`src/app/(tabs)/analytics.tsx`]

(file: `///c:/Users/kaycee/projects/bossmarvs/src/app/(tabs)/analytics.tsx`)

```
[ src/components/analytics/TimeRangeFilter.tsx ]  
(file:///c:/Users/kaycee/projects/bossmarvs/src/components/analytics/TimeRangeFilter.tsx)  
[ src/components/analytics/FinancialHealthCard.tsx ]  
(file:///c:/Users/kaycee/projects/bossmarvs/src/components/analytics/FinancialHealthCard.tsx)  
[ src/components/analytics/KeyInsightsGrid.tsx ]  
(file:///c:/Users/kaycee/projects/bossmarvs/src/components/analytics/KeyInsightsGrid.tsx)  
[ src/components/analytics/CategoryDistribution.tsx ]  
(file:///c:/Users/kaycee/projects/bossmarvs/src/components/analytics/CategoryDistribution.tsx)  
#### 📌 George's 30-Second Elevator Pitch:
```



"I built the Visual Analytics and Financial Insights module. My screen provides the user with a comprehensive financial health checkup: it calculates their net cashflow and savings rate percentage, presents a 2x2 grid highlighting their top spending category, daily average spend, biggest single purchase, and total transactions, and features a color-coded segmented breakdown bar displaying category proportions."



```
#### 🔍 Step 1: How George Queries & Filters Data by Timeframe (in analytics.tsx ) George provides  
timeframe switching ( This Month, This Week, All Time) and filters transactions accordingly:
```

TYPESCRIPT

```

const { transactions, categories } = useApp();
const [selectedRange, setSelectedRange] = useState<TimeRange>('this_month');

const periodTransactions = useMemo(() => {
  const now = new Date();
  const currentMonthPrefix = now.toISOString().slice(0, 7);

  // Week calculation (Monday to Sunday)
  const day = now.getDay();
  const diffToMonday = (day + 6) % 7;
  const monday = new Date(now);
  monday.setDate(now.getDate() - diffToMonday);
  const mondayStr = monday.toISOString().split('T')[0];

  const sunday = new Date(monday);
  sunday.setDate(monday.getDate() + 6);
  const sundayStr = sunday.toISOString().split('T')[0];

  return transactions.filter((t) => {
    if (selectedRange === 'this_month') return t.date.startsWith(currentMonthPrefix);
    if (selectedRange === 'this_week') return t.date >= mondayStr && t.date <= sundayStr;
    return true; // 'all_time'
  });
}, [transactions, selectedRange]);

```

Explanation: Tapping one of the timeframe pill buttons updates `selectedRange`. `useMemo` immediately re-runs the date filter to extract only transactions within that period.

🌈 Step 2: How George Computes Analytics Metrics in a High-Speed Single Pass George avoids multiple redundant loops by calculating income, expenses, and expense records in a single $O(N)$ pass:

TYPESCRIPT

```
const { periodIncome, periodExpenses, expenseTransactions } = useMemo(() => {
  let income = 0;
  let expenses = 0;
  const explist: typeof transactions = [];

  periodTransactions.forEach((t) => {
    if (t.type === 'income') {
      income += t.amount;
    } else {
      expenses += t.amount;
      explist.push(t);
    }
  });

  return { periodIncome: income, periodExpenses: expenses, expenseTransactions: explist };
}, [periodTransactions]);
```

Line-by-Line Explanation: Loops through `periodTransactions` exactly once. If `t.type === 'income'`, adds to `income`. If `t.type === 'expense'`, adds to `expenses` and pushes the transaction into `explist` for category grouping.

💡 Step 3: George's Key Financial Formulas & Ratios

Savings Rate Formula (in `FinancialHealthCard.tsx`) :

TYPESCRIPT

```
const netSavings = income - expenses;
const savingsRate = income > 0 ? Math.round((netSavings / income) * 100) : 0;
```

Formula:**Savings Rate Formula:**

$$\text{Savings Rate} = ((\text{Income} - \text{Expenses}) \div \text{Income}) \times 100\%$$

If income is \$2,000 and expenses are \$1,500, net savings is \$500, yielding a 25% savings rate.

Daily Average Spend Calculation:

TYPESCRIPT

```

Let daysInPeriod = 1;
if (selectedRange === 'this_month') {
  daysInPeriod = Math.max(now.getDate(), 1); // Days elapsed so far this month
} else if (selectedRange === 'this_week') {
  const dayOfWeek = now.getDay() === 0 ? 7 : now.getDay();
  daysInPeriod = Math.max(dayOfWeek, 1); // Days elapsed so far this week
}
const dailyAverage = periodExpenses / daysInPeriod;

```

Explanation: Divides total period expenses by the days that have actually elapsed so far in the current month or week, delivering an accurate burn rate.

🐛 Step 4: Category Distribution Segmented Bar (in CategoryDistribution.tsx) George draws a multi-colored segmented horizontal bar representing category proportions:

TYPESCRIPT

```

<View style={styles.segmentedBar}>
  {items.map((item) => (
    <View
      key={item.category}
      style={[
        styles.segment,
        {
          width: `${Math.max(item.percentage, 2)}%`,
          backgroundColor: item.categoryObj?.color || COLORS.primary,
        },
      ]}
    />
  ))}
</View>

```

Explanation: Uses `Math.max(item.percentage, 2)%` so that even small 1% expenses remain visible as distinct colored slivers rather than disappearing completely.

🐛 Defense Questions for George:

Q: Why did you group income and expense calculations into a single `forEach` loop?

George's Answer: "For performance optimization. Running separate `.filter()` calls for income and

expenses would iterate through the dataset multiple times. A single-pass loop computes income, expenses, and extracts the expense list in $O(N)$ linear time, keeping calculations instant even on older devices."

Q: How does the Daily Average spend handle the 1st day of the month without dividing by zero?

George's Answer: "We use `Math.max(now.getDate(), 1)`. On the 1st day of the month, `getDate()` is 1, so it divides by 1 rather than producing a NaN or infinity error."

6. Master Code Review & Defense Question Bank (Top 25 Questions)

Core Architecture & Storage Questions:

Q: Why did you use `AsyncStorage` instead of an online database like Firebase or MySQL?

Answer: "Because our app is designed to be an offline-first, private personal finance tracker. `AsyncStorage` stores data directly on the user's phone, requiring zero internet connection, zero login, and eliminating network latency entirely."

Q: How does `AsyncStorage` store data under the hood?

Answer: "It is an unencrypted, asynchronous key-value storage system that writes plain text strings to the mobile operating system's sandboxed storage partition. On Android it is backed by SQLite/RocksDB; on iOS it is backed by serialized files or SQLite."

Q: What happens if you forget to use `JSON.stringify()` when saving data to `AsyncStorage`?

Answer: "`AsyncStorage` only accepts strings. If you pass an object or array without stringifying, it will either throw a `TypeError` or write `[object Object]` as raw text, permanently corrupting your data."

Q: Why do your storage keys contain a version prefix (`@bossmarvs_v2_`)?

Answer: "Versioning storage keys prevents cached dummy data from conflicting with new data models. By incrementing to `_v2_`, the app cleanly starts with an empty slate and no residual sample records."

Q: What is the benefit of wrapping the app in `?`

Answer: "It implements React's Context API. Instead of 'prop drilling' (passing data manually down through every screen), any component in the app can call `useApp()` to read transactions, budgets, or call `save/delete` functions."

Q: Why is `loadData()` inside `useEffect` wrapped in `Promise.all`?

Answer: "Instead of loading transactions, waiting, then loading budgets, waiting, and loading categories sequentially, `Promise.all` reads all three keys from disk concurrently in parallel,

cutting cold-boot startup time by more than 50%."

Navigation & Routing Questions:

Q: How does Expo Router determine which screen to display?

Answer: "Expo Router uses file-based routing. Every file placed in `src/app/` automatically registers as a route. Files inside `(tabs)` become bottom tab screens, while `add.tsx` is configured as a modal in our root stack."

Q: What is the difference between Stack Navigation and Tab Navigation in this app?

Answer: "Tab navigation allows users to switch between the 4 main sections (Dashboard, Transactions, Budgets, Analytics) via bottom tab icons. Stack navigation manages modal transitions, sliding `add.tsx` on top of the screen and popping it off with `router.back()`."

Q: Why is the tab folder named `(tabs)` with parentheses?

Answer: "In Expo Router, enclosing a folder in parentheses creates an unnamed route group. It tells Expo Router to organize these screens inside the tab navigator without adding the word `(tabs)` to the URL route."

Q: How does `router.push('/add')` work under the hood?

Answer: "It tells the Root Stack navigator to instantiate the screen at `src/app/add.tsx` and push it onto the top of the navigation stack with a slide-up animation."

Mobile UI & Layouting Questions:

Q: How does Flexbox in React Native differ from Flexbox in standard web CSS?

Answer: "In web CSS, `flexDirection` defaults to `row` (horizontal). In React Native, `flexDirection` defaults to `column` (vertical) because mobile phones are held portrait-style."

Q: Why did you use `FlatList` for transactions instead of `ScrollView`?

Answer: "Performance and memory management. `ScrollView` renders every single record at once. `FlatList` uses list virtualization: it only renders the ~8 items currently visible on screen and recycles off-screen views, preventing memory leaks and frame drops."

Q: What does `flex: 1` achieve on container views?

Answer: "It tells the container view to expand and occupy 100% of the available remaining space on the phone's screen, adapting smoothly to any phone size or aspect ratio."

Q: Why did you centralize theme tokens in `src/constants/theme.ts`?

Answer: "For design consistency and maintainability. If we want to adjust the primary mint color or card border radius, we change one variable in `theme.ts` instead of editing dozens of separate style sheets."

Performance & Business Logic Questions:

Q: Why did you use `useMemo` in Transactions and Analytics?

Answer: "Filtering, searching, and calculating financial metrics require running loops over

`arrays. useMemo` memoizes (caches) the computed results so they only recalculate when dependencies like search text or transaction arrays change, keeping typing fluid."

Q: How does the app prevent duplicate budget entries for the same category?

Answer: "Inside `setBudget` in `AppContext.tsx`, we call `budgets.findIndex((b) => b.category === budgetData.category)`. If an existing entry is found, we update its limit in-place rather than appending a duplicate."

Q: What happens if a user spends more than 100% of their category budget?

Answer: "The visual bar width is clamped using `Math.min(percentage, 100)%` so it doesn't overflow the card. Concurrently, the status color flips to coral red (`COLORS.danger`), an 'Exceeded' badge is shown, and the Budget Alert Banner is displayed."

Q: How does your search filter handle case sensitivity?

Answer: "It converts both the user's search text and the transaction title/category/notes to lowercase using `.toLowerCase()` before checking `.includes()`. Searching for 'food' matches 'Food & Dining'."

Q: How does the Daily Average spend calculation work in Analytics?

Answer: "For 'This Month', it divides expenses by elapsed calendar days (`now.getDate()`). For 'This Week', it divides by days elapsed in the current week. This provides an accurate daily burn rate."

Q: Why is there a native `Alert.alert` confirmation before deleting records?

Answer: "To prevent accidental taps. Financial records cannot be recovered once removed from storage, so we prompt the user with a confirmation dialog before executing `deleteTransaction`."

Q: How does the category breakdown handle categories with only 1% spending?

Answer: "We use `Math.max(item.percentage, 2)%` for segment widths. This ensures that even tiny 1% expenses remain visible as small colored slivers rather than vanishing completely."

Q: How does TypeScript enhance code quality in this project?

Answer: "It enforces strict data contracts. In `src/types/index.ts`, we defined the exact fields of `Transaction` and `Budget`. If a developer passes a string where a number is expected, TypeScript flags the error at compile-time before the app runs."

Q: Why did you remove the redundant 'All Transactions' title in Tab 2?

Answer: "Because the top navigation header already displays 'Transactions'. Having 'All Transactions' immediately below it wasted vertical screen real estate. We replaced it with a live search bar and filter controls, mirroring modern financial apps."

Q: How does the app ensure user privacy?

Answer: "All data is written directly to the phone's sandboxed local storage via `AsyncStorage`. Zero financial data is ever transmitted over the network or stored on third-party cloud servers."

Q: What does the $O(N)$ single-pass loop in Analytics achieve?

Answer: "It calculates total income, total expenses, and extracts the list of expenses in a single pass through the dataset, avoiding multiple redundant array loops and keeping calculation instant even on low-end hardware."